ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP CÁ NHÂN

Môn học: Nhập môn lập trình

ĐỀ TÀI:

XÂY DỰNG TRÒ CHƠI OẲN TÙ TÌ TƯƠNG TÁC QUA CAMERA BẰNG MEDIAPIPE

Sinh viên thực hiện: Dương Tuấn Minh

Giảng viên hướng dẫn: Trần Nhật Khoa

Tháng 11 năm 2026

MỤC LỤC

1. Giới thiệu đề tài, dữ liệu đầu vào và đầu ra
1.1. Ý tưởng và mục tiêu của ứng dụng
1.2. Mô tả chi tiết dữ liệu đầu vào và đầu ra
2. Cơ sở lý thuyết và giải thuật thiết kế
2.1. Quy trình xử lý tổng thể (pipeline)
2.2. Tiền xử lý khung hình
2.3. Chức năng MediaPipe được sử dụng và cấu trúc landmark
2.4. Giải thuật hình học nhận diện trạng thái ngón tay
2.5. Thuật toán xét thắng-thua và trạng thái trò chơi
2.6. Máy trạng thái hữu hạn (FSM) điều phối thể thức thi đấu
2.7. Kỹ thuật kết xuất phông chữ TrueType và giao diện
3. Kết quả thực nghiệm
3.1. Cấu hình môi trường thực nghiệm
3.2. Kết quả nhận diện thực tế
4. Kết luận, khó khăn gặp phải và hướng phát triển
4.1. Kết luận
4.2. Những khó khăn gặp phải và giải pháp khắc phục
4.3. Hướng phát triển trong tương lai
5. Tài liệu tham khảo

1. Giới thiệu đề tài, Input và Output

1.1. Ý tưởng và mục tiêu của ứng dụng

Xuất phát từ mong muốn vận dụng kiến thức môn Nhập môn lập trình với ngôn ngữ Python vào lĩnh vực thị giác máy tính, đề tài tập trung xây dựng trò chơi oẳn tù tì tương tác trực tiếp với người dùng qua camera thời gian thực. Thay vì lựa chọn nước đi bằng chuột hoặc bàn phím truyền thống, người chơi sử dụng chính cử chỉ bàn tay trước ống kính webcam để thi đấu trực tiếp với máy tính.

Điểm nhấn của ứng dụng là việc tích hợp thể thức thi đấu best-of-7 (7 trận ăn 4 - bên nào chạm mốc 4 trận thắng trước sẽ giành chiến thắng chung cuộc với các tỷ số có thể xảy ra là 4-0, 4-1, 4-2 hoặc 4-3) lấy cảm hứng từ giải bóng rổ nhà nghề Mỹ NBA và "Đại hội Oẳn tù tì" của nhóm nhạc thần tượng Nhật Bản AKB48. Mục tiêu cốt lõi của đề tài là khai thác dữ liệu tọa độ bàn tay thô từ MediaPipe framework, sau đó tự thiết kế giải thuật hình học để phân loại chính xác 3 cử chỉ (Rock, Paper, Scissors) và xây dựng máy trạng thái hữu hạn (finite state machine - FSM) quản lý toàn bộ luồng thi đấu.

1.2. Mô tả chi tiết dữ liệu đầu vào và đầu ra

Hệ thống được thiết kế với cơ chế phân tách rõ ràng giữa dữ liệu đầu vào, dữ liệu trích xuất trung gian và kết quả đầu ra hiển thị trên cửa sổ giao diện:

Thành phần	Loại dữ liệu / Nguồn	Mô tả chi tiết trong chương trình
Đầu vào (input)	Video và bàn phím	Luồng khung hình BGR thu trực tiếp từ webcam qua cv2.VideoCapture(0) cùng các phím điều khiển: Space (bắt đầu ván mới), R (chơi lại loạt trận mới khi kết thúc), Q hoặc ESC (thoát chương trình).

Dữ liệu MediaPipe	21 điểm mốc (landmarks) trên bàn tay	Bộ nhận diện mp.solutions.hands trích xuất 21 điểm mốc giải phẫu bàn tay với tọa độ chuẩn hóa (x, y, z) trong miền giá trị [0.0, 1.0].
Đầu ra (output)	Giao diện đồ họa và điểm số	Cửa sổ trò chơi hiển thị khung xương bàn tay, cử chỉ hiện tại, bảng điểm, thanh trạng thái loạt trận, kết quả từng trận và màn hình điểm số chung cuộc.

2. Cơ sở lý thuyết và giải thuật thiết kế

2.1. Quy trình xử lý tổng thể (Pipeline)

Để đảm bảo khả năng phản hồi thời gian thực, chương trình được vận hành bên trong vòng lặp while cap.isOpened(): xử lý liên tục từng khung hình. Quy trình tổng thể trải qua 6 giai đoạn tuần tự:

1. Khởi tạo Camera (cv2.VideoCapture) và mô hình MediaPipe Hands (mp.solutions.hands)
2. Tiền xử lý ảnh: Lật gương ngang (cv2.flip) và chuyển hệ màu BGR sang RGB (cv2.cvtColor)
3. Trích xuất 21 Hand Landmarks trong hệ tọa độ chuẩn hóa qua hands.process(rgb_frame)
4. Giải thuật hình học (classify_gesture): Tính khoảng cách Euclid từ cổ tay, phân loại cử chỉ
5. Máy trạng thái hữu hạn (FSM): Điều phối các trạng thái WAITING, COUNTDOWN, RESULT, SERIES_OVER
6. Kết xuất đồ họa TrueType Font (Pillow + OpenCV) và hiển thị khung hình (cv2.imshow)

2.2. Tiền xử lý khung hình

Trước khi đưa hình ảnh vào mô hình MediaPipe Hands, mỗi khung hình thu từ camera được biến đổi qua hai bước chuẩn hóa:

- **Phép lật gương ngang:** Sử dụng hàm cv2.flip(frame, 1) để lật đối xứng khung hình qua trục tung. Thao tác này tạo hiệu ứng gương soi tự nhiên, giúp chuyển động tay của người chơi đồng bộ với hình ảnh được hiển thị.
- **Chuyển đổi không gian màu:** Do OpenCV mặc định lưu trữ ma trận điểm ảnh theo thứ tự kênh màu BGR trong khi MediaPipe yêu cầu ảnh đầu vào chuẩn RGB, chương trình thực hiện phép chuyển đổi rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) trước khi gọi hàm nhận diện.

2.3. Chức năng MediaPipe được sử dụng và cấu trúc landmark

Chương trình khởi tạo đối tượng nhận diện bàn tay thông qua câu lệnh hands = mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7, min_tracking_confidence=0.7). Việc đặt max_num_hands=1 và ngưỡng tin cậy 0.7 giúp tối ưu tốc độ xử lý khung hình và loại nhiễu từ phông nền.

Khi phát hiện bàn tay trong khung hình (results.multi_hand_landmarks), MediaPipe cung cấp danh sách 21 điểm mốc (landmarks) đánh số từ 0 đến 20, mỗi điểm chứa tọa độ chuẩn hóa (x, y) theo tỷ lệ chiều rộng và chiều cao khung hình. Để xây dựng giải thuật nhận diện, ta chọn 9 landmark quan trọng nhất:

- **Landmark 0 (cổ tay):** Đóng vai trò là điểm gốc tọa độ quy chiếu cố định của bàn tay.
- **Các landmark 8, 12, 16, 20 (đầu ngón tay):** Tương ứng với 4 đầu ngón tay: trỏ (8), giữa (12), áp út (16) và út (20).
- **Các landmark 6, 10, 14, 18 (PIP/khớp đốt ngón giữa):** Tương ứng với 4 khớp đốt ngón giữa của ngón trỏ (6), giữa (10), áp út (14) và út (18).

2.4. Giải thuật hình học nhận diện trạng thái ngón tay

Một hạn chế thường gặp ở các thuật toán cơ bản là chỉ so sánh tọa độ trục tung Y giữa đầu ngón tay và khớp ngón (tip.y < pip.y). Cách tiếp cận đó chỉ đúng khi bàn tay dựng thẳng đứng hướng lên trên, nhưng sẽ sai lệch hoàn toàn khi người chơi để bàn tay nằm ngang hay chéo, vốn là tư thế tự nhiên khi chơi oẳn tù tì.

Để giải thuật đạt tính bất biến với góc xoay bàn tay, ta xây dựng hàm classify_gesture (landmarks) dựa trên phép đo khoảng cách Euclid trong mặt phẳng 2D từ cổ tay đến đầu ngón tay so với khoảng cách từ cổ tay đến khớp giữa, với Landmark 0 đặt ở cổ tay:

Công thức khoảng cách Euclid: D(A, B) = sqrt((x2 - x1)^2 + (y2 - y1)^2).

```
def classify_gesture(landmarks):
    finger_joints = [(8, 6), (12, 10), (16, 14), (20, 18)]
    wrist = landmarks[0]
    fingers_open = []

    for tip_id, pip_id in finger_joints:
        tip = landmarks[tip_id]
        pip = landmarks[pip_id]
        dist_tip = math.hypot(tip.x - wrist.x, tip.y - wrist.y)
        dist_pip = math.hypot(pip.x - wrist.x, pip.y - wrist.y)
        fingers_open.append(dist_tip > dist_pip)

    if fingers_open == [False, False, False, False]:
        return "Rock"
    elif fingers_open == [True, True, True, True]:
        return "Paper"
    elif fingers_open == [True, True, False, False]:
        return "Scissors"
    else:
        return "Unknown"
```

Nguyên lý hoạt động và bảng ánh xạ trạng thái của hàm classify_gesture(landmarks) như sau:

- **Xác định trạng thái đơn lẻ từng ngón:**
  + Khi một ngón tay duỗi ra, khoảng cách từ cổ tay đến đầu ngón tay luôn lớn hơn khoảng cách từ cổ tay đến khớp PIP (dist_tip > dist_pip ra True).
  + Khi một ngón tay gập vào lòng bàn tay, đầu ngón tay thu gần về cổ tay hơn khớp PIP (dist_tip > dist_pip ra False).
- **Rock (búa):** 4 ngón (trỏ, giữa, áp út, út) đều gập lại => fingers_open == [False, False, False, False].

- **Paper (bao):** 4 ngón đều mở => fingers_open == [True, True, True, True].
- **Scissors (kéo):** Ngón trỏ và ngón giữa mở, ngón áp út và ngón út gập lại => fingers_open == [True, True, False, False].
- **Unknown (không hợp lệ):** Mọi tổ hợp trạng thái ngón tay khác ngoài 3 mẫu chuẩn trên đều trả về "Unknown" nhằm ngăn chặn nhận diện sai khi người dùng đang chuyển tiếp cử chỉ.

2.5. Thuật toán xét thắng-thua và trạng thái trò chơi

Sau khi trích xuất được cử chỉ của người chơi và nước đi ngẫu nhiên của máy tính qua random.choice(["Rock", "Paper", "Scissors"]), hàm get_winner(player, cpu) thực hiện đối chiếu luật chơi oẳn tù tì:

```
def get_winner(player, cpu):
    if player in ["Unknown", "No hand detected"]:
        return "Invalid move"
    if player == cpu:
        return "Draw!"
    if (player == "Rock" and cpu == "Scissors") or \
       (player == "Paper" and cpu == "Rock") or \
       (player == "Scissors" and cpu == "Paper"):
        return "You win!"
    else:
        return "CPU wins!"
```

Hàm get_series_banner(p_score, c_score) tự động tính toán số thứ tự ván đấu hiện tại theo công thức game_num = p_score + c_score + 1. Vì các ván hòa ("Draw!") hoặc nước đi lỗi ("Invalid move") không làm tăng điểm của hai bên, tổng điểm p_score + c_score phản ánh chính xác số ván đã phân định thắng bại. Đặc biệt, khi tỷ số là 3-3, thông điệp "GAME 7 | SERIES TIED 3-3 | WIN OR GO HOME" sẽ hiện lên, tăng thêm phần kịch tính cho trò chơi.

2.6. Máy trạng thái hữu hạn (FSM) điều phối thể thức thi đấu

Do vòng lặp camera chạy liên tục ở tốc độ khoảng 30 khung hình/giây (fps), nếu chương trình chấm điểm trực tiếp trên từng khung hình thì máy tính sẽ đổi nước đi liên tục và kết thúc trận đấu nhanh chóng. Để giải quyết vấn đề đồng bộ thời

gian thực, ta thiết kế máy trạng thái hữu hạn (FSM) gồm 4 trạng thái được điều khiển qua biến state và hằng số TARGET_WINS = 4:

Trạng thái	Logic xử lý và sự kiện chuyển tiếp	Thông điệp trên UI
WAITING	Trạng thái khởi đầu khi mở ứng dụng. Chờ người chơi nhấn phím Space để ghi nhận mốc thời gian countdown_start = time.time() và chuyển sang COUNTDOWN.	"GAME 1 OF 7 | SERIES TIED 0-0" và lời nhắc "Press Space to start"
COUNTDOWN	Tính thời gian đếm ngược 3 giây: remaining = 3 - int(time.time() - countdown_start). Khi remaining <= 0, khóa cử chỉ hiện tại vào biến locked_player_choice, tạo nước đi cpu_choice, gọi get_winner() và cộng điểm cho bên thắng. Nếu một bên đạt TARGET_WINS (4 điểm) thì chuyển sang SERIES_OVER, ngược lại sang RESULT.	Khi remaining > 0: "Shoot in 3…" (rồi 2 và 1)
RESULT	Hiển thị nước đi đã khóa của hai bên và kết quả ván đấu (round_result). Chờ người chơi nhấn phím Space để bước vào nhịp đếm ngược của ván kế tiếp.	"You: <choice> | CPU: <choice>" cùng kết quả: "You win!" / "CPU wins!" / "Draw!" / "Invalid move" và lời nhắc: "Press Space for next game"

SERIES_OVER	Kích hoạt khi player_score == 4 hoặc cpu_score == 4. Tính tổng số ván đã đấu total_games = player_score + cpu_score (nhận giá trị 4, 5, 6 hoặc 7). Vẽ bảng tổng kết bán trong suốt và lắng nghe phím [R] (đặt lại điểm 0-0, chuyển về COUNTDOWN) hoặc [Q]/[ESC] (thoát).	Tiêu đề thắng: "SERIES WINNER: YOU WIN 4-x (IN N GAMES)!" Tiêu đề thua: "SERIES OVER: CPU WINS 4-x (IN N GAMES)!" Ghi chú phụ: "CLEAN SWEEP!" / "GAME 7 VICTORY!" / "SERIES CLINCHED!" / "SWEPT BY CPU!" / "GAME 7 LOST!" / "SERIES LOST!" Điều khiển: "Press [R] to play again" và "Press [Q] or [ESC] to exit"

2.7. Kỹ thuật kết xuất phông chữ TrueType và giao diện

Để khắc phục nhược điểm phông chữ nét đứt độ phân giải thấp của hàm cv2.putText() mặc định trong OpenCV, chương trình xây dựng hệ thống hiển thị văn bản tùy chỉnh thông qua hai hàm get_windows_font(size) và draw_text(frame, text, pos, size, color, center):

- **Cơ chế nạp và lưu đệm phông chữ:** Hàm get_windows_font(size) tự động dò tìm các tệp phông chữ TrueType trên Windows theo thứ tự ưu tiên: bahnschrift.ttf, segoeuib.ttf, arialbd.ttf và consolab.ttf. Các phông chữ đã tải được lưu vào từ điển FONT_CACHE để tránh đọc lại từng khung hình.
- **Kết xuất chữ sắc nét kèm bóng đổ và cơ chế fallback:** Hàm draw_text() chuyển khung hình sang định dạng Pillow, tính toán kích thước hộp văn bản bằng hàm draw.textbbox() để căn giữa chính xác khi tham số center=True, đồng thời vẽ một lớp bóng đổ màu đen lệch (+2px, +2px) sau chữ chính giúp giao diện dễ đọc trong mọi điều kiện ánh sáng nền. Nếu môi trường thiếu thư viện Pillow, hàm tự động chuyển về hệ thống phông chữ dự phòng cv2.FONT_HERSHEY_DUPLEX có khử răng cưa cv2.LINE_AA.

- **Hiệu ứng bảng kết quả bán trong suốt:** Ở trạng thái SERIES_OVER, chương trình sao chép khung hình hiện tại (overlay = frame.copy()), vẽ hình chữ nhật màu tối (15, 15, 15) và kết hợp với khung hình gốc bằng hàm cv2.addWeighted(overlay, 0.82, frame, 0.18, 0, frame) để làm nổi bật bảng kết quả chung cuộc mà vẫn nhìn thấy chuyển động camera phía sau.

3. Kết quả thực nghiệm

3.1. Cấu hình môi trường thực nghiệm

Ứng dụng được triển khai và kiểm thử thực tế trên môi trường ảo với bộ thông số phiên bản tương thích ổn định:

- **Phiên bản Python:** Python 3.11.9 (64-bit trên hệ điều hành Windows).
- **Lệnh cài đặt các thư viện phụ thuộc:** pip install "mediapipe==0.10.21" "numpy<2.0.0" "opencv-python<5.0.0"

3.2. Kết quả nhận diện thực tế

Qua quá trình chạy thử nghiệm với nhiều tư thế đặt tay và khoảng cách khác nhau trước webcam, giải thuật khoảng cách Euclid trong hàm classify_gesture() phân biệt chính xác cả 3 cử chỉ Rock, Paper, Scissors, đồng thời phát hiện trạng thái Unknown hoặc No hand detected. Thể thức thi đấu vận hành đúng luật ở mọi kịch bản tỉ số (4-0, 4-1, 4-2 và 4-3).

4. Kết luận, khó khăn gặp phải và hướng phát triển

4.1. Kết luận

Đề tài đã hoàn thành mục tiêu xây dựng ứng dụng trò chơi oẳn tù tì tương tác qua camera theo thời gian thực bằng ngôn ngữ Python, kết hợp thư viện OpenCV, Pillow và MediaPipe Hands. Chương trình không chỉ dừng lại ở việc thu thập dữ liệu điểm mốc bàn tay từ MediaPipe mà đã tự xây dựng trọn vẹn giải thuật hình học tính khoảng cách Euclid để phân loại cử chỉ, kết hợp máy trạng thái hữu hạn tái hiện thể thức best-of-7.

4.2. Những khó khăn gặp phải và giải pháp khắc phục

- **Khó khăn 1: Xung đột phiên bản Python và thư viện phụ thuộc:** Khi khởi chạy ban đầu trên môi trường Python 3.14.8, chương trình gặp lỗi AttributeError: module 'mediapipe' has no attribute 'solutions' và ModuleNotFoundError: No module named 'mediapipe.python' do các gói nhị phân C++ của nhánh mp.solutions chưa hỗ trợ Python 3.14, đồng thời xung đột với cấu trúc mảng của NumPy 2.x. Vấn đề này được khắc phục bằng cách thiết lập môi trường ảo (venv) trên Python 3.11.9 và cố định phiên bản tương thích: mediapipe==0.10.21, numpy<2.0.0 và opencv-python<5.0.0.
- **Khó khăn 2: Sai số nhận diện khi xoay nghiêng cổ tay:** Trong thực tế, người chơi thường có thói quen giữ bàn tay nằm ngang khi nhịp tay khiến phương pháp so sánh tọa độ trục tung Y bị sai lệch. Việc thiết kế công thức so sánh khoảng cách Euclid từ cổ tay (Landmark 0) đến đầu ngón tay và khớp PIP bằng hàm math.hypot() đã giúp giải thuật hoạt động ổn định ở mọi góc xoay bàn tay trên mặt phẳng 2D.
- **Khó khăn 3: Hiện tượng nhòe chuyển động và ánh sáng yếu:** Khi người chơi vẩy tay quá nhanh đúng vào thời điểm kết thúc đếm ngược (remaining <= 0) hoặc chơi trong phòng thiếu sáng, khung hình bị nhòe khiến MediaPipe trả về trạng thái “No hand detected” hoặc “Unknown”. Vấn đề này đã được xử lý trong hàm get_winner() bằng cách trả về thông báo “Invalid move” và giữ nguyên tỉ số hiện tại để người chơi đấu lại.

4.3. Hướng phát triển trong tương lai

- **Nâng cấp trí tuệ cho máy:** Thay vì sử dụng hàm random.choice() thuần túy, có thể ghi nhớ lịch sử ra dấu của người chơi trong các ván trước để áp dụng mô hình chuỗi Markov dự đoán xu hướng tâm lý của người chơi ở những trận quyết định, đặc biệt là Game 7.
- **Hỗ trợ 2 người chơi đối kháng trực tiếp:** Tăng max_num_hands thành 2 và chia đôi khung hình camera thành hai vùng trái, phải để hai người chơi có thể thi đấu trực tiếp với nhau.
- **Tích hợp âm thanh:** Sử dụng thư viện pygame.mixer hoặc winsound để phát hiệu ứng âm thanh đếm ngược 3 giây, tiếng còi hết trận và nhạc hiệu chiến thắng.

5. Tài liệu tham khảo

[1] Google, "MediaPipe Solutions Guide," Google AI for Developers. Địa chỉ: https://ai.google.dev/edge/mediapipe/solutions/guide (Truy cập: 11/2026)

[2] OpenCV, "OpenCV-Python Tutorials," OpenCV Documentation. Địa chỉ: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html (Truy cập: 11/2026)

[3] Pillow, "ImageFont and ImageDraw Module Documentation," Pillow (PIL Fork) Documentation. Địa chỉ: https://pillow.readthedocs.io/en/stable/reference/ImageFont.html (Truy cập: 11/2026)